WORLDFORGE OS

The Comprehensive Trust & Governance Architecture

Master Manual

A governance engine wearing a creative-pipeline UI:
its real product is an unfakeable decision record.

163,940-byte single-file artifact · zero runtime dependencies
48 verification lanes GREEN · 133/133 smoke assertions
9 hostile chaos-probe families · zero findings

Compiled 2026-07-20 · Ratification authority: Irfan

CONTENTS

What This Manual Covers

> **Provenance of this document.** Every architectural claim below was read directly from the extracted source tree (`src/`, `test/`, `scripts/`, `build.mjs`) and the ratified records in `docs/decision_log.md` and `docs/SYSTEM_IDENTITY.md`. Every number in the Verification Scoreboard (Chapter 4) is copied verbatim from measured telemetry in the archive — `work/matrix-48-run.json` and `work/jarvis-run.json` — not estimated. Where the system documents a limit (mock gateway, local job-runner, keyless chain), this manual repeats the limit rather than papering over it.

CHAPTER 1

Executive Summary

WorldForge OS presents itself as a 3D creative-production pipeline tracker — ten stages, three gates, a budget ledger, an asset library, eight guilds. That is its surface. Structurally it is a **governance kernel**: a small, hard, dependency-free state machine whose actual product is a **decision record that cannot be quietly falsified**, wrapped in a UI pleasant enough that people will actually use it.

The entire tree is organised around one asymmetry: *mutations are cheap, records are expensive to fake*. Every architectural decision in the repository — the single-file artifact, the injected seams, the frozen data shapes, the mutation FIFO, the SHA-256 hash chain, the boot self-check, the notary-style build, the chaos harness — exists to preserve the trustworthiness of that record on hostile ground: a browser, someone else's storage tier, an offline `file://` double-click, a spoofed payment webhook.

Where the system stands today

48 / 48	133 / 133	0	163,940 B
verification lanes GREEN	smoke assertions passing	chaos findings (9 families)	single-file artifact

Measured on one machine from work/matrix-48-run.json (status GREEN) and the six unit suites. Zero runtime dependencies; the artifact refuses to boot if a single byte of its own executable payload has been altered.

The one-sentence thesis

> *"WorldForge OS is a governance engine wearing a creative-pipeline UI: its real product is an unfakeable decision record, and every architectural choice in the tree — single-file artifact, injected seams, frozen shapes, fail-closed law, chaos suite — exists to keep that record trustworthy on hostile ground."*

The remainder of this manual makes that thesis operational: Chapter 2 lays out the four nested trust boundaries that give the record its integrity; Chapter 3 dissects the four mechanisms that defend it under concurrency, corruption, and adversarial input; Chapter 4 is the live evidence that those mechanisms hold; and Chapter 5 is an engineering vision — explicitly proposal, not record — for turning the substrate into a product.

CHAPTER 2

Core Philosophy — Four Nested Trust Boundaries

Read the repository as syntax and you see a project tracker. Read it as *structure* and you see four nested trust boundaries, each less trusted than the one it wraps. Authority contracts inward; anything outward must ask permission through a mechanically-enforced seam. This is the ontology on which every guarantee rests.

01 THE KERNEL

worldforge-kernel.v1.1.js — owns mutation truth

The innermost boundary and the only tier permitted to change state. Its loop is **freshen → mutate → ONE persist**. Decisions are append-only, capped, and written *atomically with* the state change they justify — you cannot move a stage and forget to say why. Gate semantics fire on *leaving* a stage: the system taxes exits, not entries, because regret is discovered late. Roughly a third of its 257 lines exist purely to guarantee that a decision record lands in the same storage write as the change it explains.

02 THE EXT

wfkernel-p2-ext.v1.2.js — the constitution's amendments

An additive mixin only; the kernel file is never edited by it. It contributes the governance vocabulary — promotion, locking, budget, UFDM export — and, after the chaos pass, the **mutation FIFO** (D-2026-07-19-02) that makes in-process truth single-threaded. Its maintainer law is exact: a queued method must never call another queued method (deadlock); un-queued callers may call queued ones.

03 THE COMPONENTS

forge, roster, visual surfaces — deliberately powerless

The presentation tier renders and nothing more. Forge cannot reach the kernel except through the adapter, and `build.mjs --gate` enforces this *mechanically, not by convention*. The override ceremony is the clearest statement of identity in the codebase: the *easy* path is refusal, and the bypass costs a name and a reason, permanently, rendered next to every clean pass. Overrides are not exceptions to governance — they are its most important records.

04 THE NOTARY (BUILD)

build.mjs — a notary, not a compiler

The outermost boundary. No minifier, no transpiler, no tree-shaker. It performs a deterministic concat in a hand-declared order, splices the single-source kernel between `WF:KERNEL` markers (D-2026-07-19-01), embeds a content hash, and can re-derive it to prove freshness. It stamps per-block SHA-256 hashes so the artifact refuses to boot if its own payload was altered (D-2026-07-19-06). The artifact can prove what it is.

The two-law discipline

Every seam obeys one of two laws, and the codebase never confuses them:

FAIL-CLOSED	Everywhere money or governance is involved. Corrupt rows abort billing; unrecordable overrides do not happen; an ambiguous gateway response is *unpaid*, never assumed paid. When the system cannot be sure it is safe, it stops.
FAIL-LOUD	Everywhere recovery is possible. Corrupt entries skip with events and console noise, never silently; a corrupt primary read auto-rolls-back with provenance. When the system can proceed safely, it does — but it always leaves a trace.

Frozen invariants

Violating any of these is an identity change, not a bug:

- One storage write per mutation; the record lands with the change.
- Ledger entries carry `cost` — `amount` is a detected, rejected shape.
- Fail-closed everywhere money or governance is involved.
- Fail-loud everywhere recovery is possible.
- Vanilla JS + WAAPI, UMD, zero runtime dependencies (D-2026-07-18-04). The artifact must survive a `file://` double-click on an offline machine.
- The chaos suite is part of the product. A fix without a probe is a rumor.

CHAPTER 3

Technical Deep-Dive

Four mechanisms defend the record under the conditions that would otherwise corrupt it: concurrency, corruption-at-rest, historical tampering, and adversarial payment input. Each is a self-contained module with its own smoke suite; each was added in response to a specific, reproduced failure.

3.1 The Mutation FIFO

Source: wfkernel-p2-ext.v1.2.js · Ratified D-2026-07-19-02 · Chaos findings 3 & 4

The kernel's `freshen()` replaces the in-memory state object before a mutation is applied. Under concurrency this opens a lost-write window: a racing `advance()` and `updateBudget()` each hold a reference to a different snapshot, and the last persist silently wins. The fuzz harness reproduced it (finding 3); a browser first-use race reproduced it live (finding 4) — an in-flight `loadProjects()` wholesale-replaced the array and wiped a concurrent `createProject()`; the user saw ‘Save failed’ and a new project vanished.

The immunity is a single **in-process FIFO** installed by the ext, wrapping all twelve state-replacing operations so they serialise through one queue. Cross-client semantics are unchanged (freshen, last-writer-wins); only *in-process* truth becomes single-threaded. The queue seam is exposed to sibling extensions as `kernel._p2Enqueue`, which the monetization layer joins so meter writes cannot re-create the race. The maintainer law — a queued method must never call another queued method — is what keeps the single thread from dead-locking on itself.

3.2 Gap 1 — Automated State Recovery

Source: src/wf-recovery.v1.js · Ratified D-2026-07-19-04 · closes gap G1

Before this tier, corruption-at-rest was detected loudly (kernel 1.1.1) but the data was simply lost — **detection without restitution**. G1 was sequenced first because it is the only gap where the system *knew* it had lost user data and could do nothing about it. The layer closes it at the storage seam: it wraps the injected adapter, so the kernel, ext, and monetization tiers are untouched.

WRITE	A guarded key's value must parse as a JSON object or the write **throws** — fail-closed, a corrupt transaction never lands. After the primary persists, a hash-stamped shadow copy is written: `{ h, v, ts }`, the last pristine state-hash the system can always fall back to.
READ	A guarded primary that fails to parse triggers auto-rollback: the shadow's hash is re-verified first (**a tampered shadow is refused, never restored**), the primary is rewritten from the shadow, the event is loud (console + `onEvent`), and the caller transparently receives the recovered state. No shadow, or shadow tampered → loud null. Recovery never guesses.

*Hashing is FNV-1a 32-bit, synchronous and dependency-free: this tier provides **integrity** provenance (bit-rot, truncation, partial writes), not cryptographic tamper-proofing. That distinction is deliberate — cryptographic chaining is the next mechanism.*

3.3 Gap 2 — The SHA-256 Event Chain

Source: src/wf-event-chain.v1.js · Ratified D-2026-07-19-05 · closes PROV-1

The decision log is append-only *by discipline*, not by math — a hostile storage tier could rewrite history. G2 makes the governance ledger **tamper-evident**: every appended decision is hash-chained to its predecessor via `_chain = { seq, prev, hash }`, so altering, reordering, deleting, or inserting any historical record breaks `verifyEventChain()` at the exact index of tampering. It was verified against Node's crypto on 13 vectors, including multi-block inputs and surrogate pairs.

> **Why a hand-written SHA-256.** The module contains 158 lines of pure-JS bit-twiddling for one reason: the kernel's record-append path is *synchronous*, and browser SubtleCrypto is async-only, so it cannot back a sync append. The system would rather implement a hash function than let a governance record be appended without provenance. The chain fields are excluded from the digest, so a record's hash never depends on itself.

Honest scope (frozen with the decision): this is keyless tamper-*evidence*. It proves history was not altered by anyone who does not also rewrite the whole chain forward; an attacker who controls all bytes could recompute the entire chain. Keyed, signed authenticity is a further step, tracked as marker `PROV-2`. The value delivered today is that *any partial edit is caught*.

3.4 The MON-1 Server Payment Bridge

Source: src/wf-payment-bridge.v1.js (server-side) + src/wf.monetization.v1.js (contract) · D-2026-07-19-03

The payment bridge is **server-side only** and deliberately absent from the `build.mjs` bundle manifest — the offline artifact's isolation is untouched. It is the verification and provenance half of a payment integration, and it is careful to be no more than that.

- **verifySignature** — Stripe-scheme webhook authentication: header `t=<unix>,v1=<hex>`, HMAC-SHA256 over `${t}.${rawBody}`, constant-time compare, bounded timestamp tolerance (replay window, 300 s).
- **normalizeProcessorEvent** — translates processor vocabulary into the frozen ledger vocabulary: money lands as `cost` in major units; `amount` is processor dialect and never survives into a record.
- **ingest** — appends a `payment-verified` governance record through the kernel's `recordDecision` channel, so it is hash-chained by the G2 event chain and persisted exactly once.

> **What it is NOT.** It moves no money, calls no processor API, and holds no card data. It only proves ‘a correctly-signed webhook arrived and was recorded immutably’. The default monetization gateway is a **MOCK**; no real payment processing exists anywhere in this codebase. Charging remains behind the `MON-1` real-gateway decision, which has not been recorded.

Its fail-closed law is enumerated as a probed error register — every clause is exercised in `test/payment-bridge-smoke.mjs` and fuzzed in the chaos harness:

Code	Fail-closed clause
`PAY-01`	No secret configured — nothing verifies
`PAY-02`	Malformed signature header
`PAY-03`	Timestamp outside tolerance — replay refused
`PAY-04`	HMAC mismatch
`PAY-05`	Event without id / type
`PAY-06`	Ambiguous money — never guessed
`PAY-07`	Duplicate event id — idempotent, ledger untouched

The server-governance telemetry in the archive shows the whole path working end-to-end: a verified `evt_demo_001` (payment_intent.succeeded, cost 49.99 USD) recorded as a hash-chained `payment-verified` decision at chain seq 0.

CHAPTER 4

Operational Verification Scoreboard

This chapter is the live evidence that the mechanisms of Chapter 3 hold. Every figure is copied from measured telemetry in the archive. The headline is the 48-lane verification matrix; beneath it sit the six unit suites (133 assertions) and the chaos immunity record.

> **Honesty contract (from scripts/agent-matrix-48.mjs).** The 48 ‘lanes’ are a **local parallel job runner on one machine** — 48 real micro-jobs (real subprocesses and real file assertions reporting real exit codes), *not* 48 compute nodes and not a distributed or decentralized architecture. No model calls, no network, no remote compute. WorldForge OS remains a single self-contained HTML artifact by design. Cluster names are organisational labels for related jobs.

4.1 The 48-lane matrix — by cluster

Cluster	Focus	Lanes	Green
Engine Core	kernel / ext / billing / recovery contracts + syntax + bundle hash	10	**10 ✓**
Crypto & Red-Team	event chain, payment webhook bridge, chaos harness, scope confinement, suite integrity	12	**12 ✓**
Perf & WAAPI	render-path syntax, artifact weight budget, WAAPI / rAF leak checks	10	**10 ✓**
Hygiene & Linters	rules matrix freshness, roster + schema lint, TODO sweep, orchestrator syntax	8	**8 ✓**
Branding & Telemetry	collateral presence, identity thesis, ratified decision log, telemetry parse	8	**8 ✓**
TOTAL	**status GREEN**	**48**	**48 ✓**

*Run telemetry (work/matrix-48-run.json): concurrency 8 · wall 3,675 ms · serial-equivalent 20,642 ms · **5.62× speed-up** · artifact 163,940 B · status **GREEN**. Every ✓ is an exit-code-0 you can reproduce by running the job yourself.*

4.2 The 133 smoke assertions — six unit suites

Suite	Covers	Passed
`kernel-smoke.mjs`	kernel contract: atomic record+mutation, gate-on-leaving, loud corrupt-skip	**30 / 30**
`p2-ext-smoke.mjs`	ext v1.2.1: 21 baseline + 6 read-seams + 4 chaos regressions (incl. FIFO)	**31 / 31**
`payment-bridge-smoke.mjs`	MON-1 bridge: HMAC auth, replay window, normalization, idempotency (PAY-01..07)	**25 / 25**
`monetization-smoke.mjs`	billing contract: meter writes, corrupt-row abort, tamper-checked invoicing	**20 / 20**
`event-chain-smoke.mjs`	G2 chain: SHA-256 vectors, break-index on edit / delete / reorder / insert	**16 / 16**

`recovery-smoke.mjs`	G1 recovery: shadow stamping, auto-rollback, forged-shadow refusal	**11 / 11**
TOTAL	**six suites, zero failures**	**133 / 133**

30 + 31 + 25 + 20 + 16 + 11 = 133. Counts cross-checked against the jarvis-run and matrix-48 telemetry tails in the archive.

4.3 Chaos immunity — five reproduced failures, all fixed

The chaos suite is treated as part of the product: *a fix without a probe is a rumor*. Every failure below was reproduced, patched, and turned into a permanent regression probe. Nine probe families now run with zero findings.

Sev	Failure mode	Immunity
CRIT	One null row in budget_ledger crashed getBudgetSummary() — whole budget UI dead from one corrupt row	Corrupt rows counted & console-errored, never fatal
HIGH	reserveBudget wrote {amount} — the frozen-shape violation; reservations invisible, burnstrip rendered NaN	Writes {cost, ts}; legacy amount honored on read
HIGH	Racing advance() vs updateBudget() lost ledger writes — freshen replaced the object mid-flight	Global in-process mutation FIFO (12 ops)
HIGH	First-use race: in-flight loadProjects() wiped a concurrent createProject() — new project vanished	Same FIFO — load / create serialized with mutators
MED	Corrupt project entry skipped silently on load — a project vanishes with no trace (fail-open)	Loud skip: console.error + event + corrupt count (kernel 1.1.1)

Probes that survived unpatched: overdraw refusal, chained kernel+ext eventing, hostile-listener containment, and global scope confined to exactly the six expected UMD names. Boot self-check negative-tested live: kernel-block and bundle-block tampers both refused with block-precise diagnostics.

CHAPTER 5

The Strategic Roadmap

> **Status of this chapter: proposal, not record.** Everything below is engineering judgment developed from the review. Nothing here has been ratified, and several items would require decision records in `docs/decision_log.md` before implementation. It is kept distinct from the measured chapters on purpose — the whole system exists to keep proposal and record from being confused.

5.1 The strategic observation

The most valuable thing in this repository is not the pipeline tracker. It is the **governance substrate**: kernel + ext + recovery + event chain + boot self-check + chaos harness — roughly 55 KB of dependency-free JavaScript providing atomic record-with-mutation, tamper-evident history, self-healing storage, and artifact self-verification, with a documented seam architecture and an honest statement of its own limits. That substrate has *nothing to do with film production*. The ten stages, three gates, and eight guilds are configuration.

Any domain where an irreversible decision needs an attributable, verifiable record has the same shape:

- Clinical-trial protocol deviations
- Model-deployment approvals in regulated ML
- Change-advisory boards in regulated infrastructure
- Financial close and journal-entry approvals
- Legal matter management
- Aviation and industrial maintenance sign-offs
- Academic research integrity and provenance
- Supply-chain custody transfers

> **Recommendation P0 — extract `@worldforge/governance-kernel`.** The seams already exist: pipeline, roster, rules, storage, actor, and digest are all injected. Extraction is packaging and documentation, not re-architecture. WorldForge OS then becomes the reference implementation and the flagship demo. This is the single highest-leverage move available — the codebase has already done ninety percent of the work.

5.2 Closing the tracked gaps — proposed order

Marker	Move	Why it matters
`TEN-1`	Multi-tenant key derived at kernel construction, enforced at the adapter via a new --tenantcheck gate	The commercial unlock — nothing enterprise-shaped ships without it
`CAS-1`	Optional setIfVersion capability the recovery wrapper probes for; degrade to LWW and say so via kernel.concurrencyMode	The correctness gap — make the guarantee legible, never fake it

PROV-2	Optional signing seam over the existing chain head (Ed25519 in-browser, HSM/KMS server-side)	Keyed authenticity; keyless chain stays the default fallback
ACT-1	pipeline.actorPolicy: free | roster | roster-or-human, default unchanged	Enterprises wire roster to SSO; solo users keep free text

5.3 New capabilities worth building

- **Decision-record archival** — roll evicted decisions into a hash-anchored archive instead of discarding at cap-100. Removes the only place the record is silently made less complete (~40 lines, a correctness issue not a feature).
- **The Audit Bundle** — a single self-contained HTML file carrying the full decision chain, an embedded browser verifier, the exact kernel/artifact hashes, and a plain-language integrity verdict. The natural commercial artifact of this architecture; needs no new trust machinery.
- **Server-side recovery parity** — wrap server.js fileStorage with WFRecovery and make flush() crash-atomic (tmp → fsync → rename). Closes a real truncation window in ~5 lines.
- **Gate analytics** — override rate per gate, time-in-stage, reversal depth. The key insight: override rate is a signal about the *gate*, not the people — it distinguishes an undisciplined team from a mis-placed checkpoint.
- **Storage adapters as a published set** — localStorage, IndexedDB, Node file, S3/R2, Postgres, in-memory — each with its own smoke suite. Adapters are the adoption surface; IndexedDB / S3 / Postgres also unlock real CAS.
- **Time-travel replay** — kernel.replayTo(seq) returning a read-only projection of state as it stood at any decision index. The append-only, hash-chained data model already supports it; nothing exposes it yet.

5.4 The deliberate anti-roadmap

Discipline is easier to lose than to build. Three things this system should refuse to do, because each would trade away the property that makes it valuable:

Do NOT add a framework	The zero-dependency constraint is what makes the artifact auditable and offline-capable. Every dependency is a trust delegation.
Do NOT minify	Byte-shredding would fight the audit discipline that caught these bugs. The comments are load-bearing — several encode invariants that exist nowhere else.
Do NOT go multi-file	One file that hashes itself is the trust model. Splitting it forfeits boot self-verification.

APPENDIX

Decision Log, Error Register & Inventory

A.1 Ratified decision records

Append-only. Ratification authority: Irfan. A decision with real weight gets recorded, not just decided (Rule 7).

Record	Ratified decision
`D-…-18-04`	Engineering language: Vanilla JS + WAAPI, UMD, zero build-chain dependencies.
`D-…-19-01`	Kernel single-source splice — build.mjs folds the kernel into the determinism hash.
`D-…-19-02`	In-process mutation FIFO — all state-replacing ops serialize through one queue.
`D-…-19-03`	Monetization contract v1 — frozen billing contract, mock gateway, bundle-excluded.
`D-…-19-04`	Recovery tier at the storage seam — hash-stamped shadows, loud auto-rollback.
`D-…-19-05`	Cryptographic event chain (G2 / PROV-1) — SHA-256 hash-chained governance records.
`D-…-19-06`	Artifact boot self-check (G5 / SELF-1) — refuses to boot on per-block hash mismatch.

A.2 Open markers (not yet ratified)

`MON-1` real-gateway decision · `PROV-2` keyed/signed authenticity · `TEN-1` multi-tenant prefixes · `CAS-1` storage-tier compare-and-swap · `PIPE-1` · `ACT-1` · `UFDM2-Q1–Q3` · §5–6 browser/a11y release passes (Irfan-side).

A.3 Measured inventory

Metric	Value	Source
Shipped artifact size	**163,940 B**	`matrix-48-run.json`
Monolith baseline	**653,674 B**	`chaos-report`
Reduction vs monolith	**~74.9%**	`branding regen`
Runtime dependencies	**0**	`D-…-18-04`
Global UMD names	**6 (confined)**	`scope gate`
Matrix wall time / speed-up	**3,675 ms · 5.62×**	`matrix-48-run.json`

End of manual. Compiled locally with ReportLab from the WorldForge OS full source archive. This document reproduces the system's own stated limits — the payment gateway is a mock, the 48 lanes are a local job runner, and the event chain is keyless tamper-evidence — because a manual that overstated the guarantees would violate the very identity it documents.